Լաբորատոր աշխատանք 8

Տեսական մաս (Համառոտ)

- **Ինկապսուլյացիա (Encapsulation):** Դասի տվյալները (stationName, workDays, dailyAverageProduction, lossPercentage) պահվում են private տիրույթում, իսկ մուտքն ու փոփոխումը կատարվում են public setter/getter մեթոդների միջոցով:
- **Կոնստրուկտորների գերբեռնում:** Ստեղծվել է 3 կոնստրուկտոր՝ լռելյայն (առանց պարամետրերի), 3 պարամետրով (կորուստը՝ 0) և 4 պարամետրով:
- **Տվյալների ստուգում:** Օրերի քանակը և արտադրանքը պետք է լինեն դրական, իսկ կորուստների տոկոսը՝ 0 - 25% միջակայքում: Անվավեր տվյալների դեպքում վերագրվում են լռելյայն արժեքներ և տպվում է զգուշացում:

2. Խնդրի պահանջը

- **Դասի անունը:** SolarStation
- **Տվյալներ (private):** Կայանի անուն, օրերի քանակ, օրական միջին արտադրանք (կՎտ-ժ), կորուստների տոկոս (0-25%)
- **Հաշվարկային մեթոդներ:**
    1. *Համախառն արտադրանք* = օրեր × օրական արտադրանք
    2. *Օգտակար արտադրանք* = համախառն $\times \left(1 - \frac{\text{տոկոս}}{100}\right)$
- **Ստուգիչ օրինակ:** Մուտք՝ 20 օր, օրական 50 կՎտ-ժ, կորուստ 10% → Համախառն՝ 1000 կՎտ-ժ, Օգտակար՝ 900 կՎտ-ժ:
- **Ջանգվածի մշակում:** Ցուցադրել ամենամեծ օգտակար արտադրանքով կայանը և բոլոր կայանների օգտակար արտադրանքների գումարը:

```
#include <iostream>

#include <string>


using namespace std;


class SolarStation {

private:
```

```
    string stationName;
    int workDays;
    double dailyAverageProduction; // կՎտ-ժ
    double lossPercentage;         // 0 - 25%

public:
    // 1. Կոնստրուկտորներ
    // Լռելյայն կոնստրուկտոր
    SolarStation() {
        stationName = "Unknown";
        workDays = 1;
        dailyAverageProduction = 10.0;
        lossPercentage = 0.0;
    }

    // 3 պարամետրով կոնստրուկտոր (կորուստները՝ 0)
    SolarStation(string name, int days, double dailyProd) {
        setStationName(name);
        setWorkDays(days);
        setDailyAverageProduction(dailyProd);
        lossPercentage = 0.0;
    }

    // 4 պարամետրով կոնստրուկտոր (բոլոր տվյալները)
    SolarStation(string name, int days, double dailyProd, double loss) {
        setStationName(name);
        setWorkDays(days);
        setDailyAverageProduction(dailyProd);
```

```
        setLossPercentage(loss);
    }

    // Setter-ներ (սխալ/անվավեր տվյալների ստուգմամբ)
    void setStationName(string name) {
        if (name.empty()) {
            stationName = "Unknown";
            cout << "Զգուշացում: Կայանի անունը դատարկ էր, սահմանվեց " << stationName << "-ով:\n";
        } else {
            stationName = name;
        }
    }

    void setWorkDays(int days) {
        if (days <= 0) {
            workDays = 1; // լռելյայն արժեք
            cout << "Զգուշացում: Օրերի քանակը անվավեր է (" << days << "), սահմանվեց լռելյայն՝ 1:\n";
        } else {
            workDays = days;
        }
    }

    void setDailyAverageProduction(double dailyProd) {
        if (dailyProd <= 0) {
            dailyAverageProduction = 10.0; // լռելյայն արժեք
            cout << "Զգուշացում: Օրական արտադրանքը անվավեր է (" << dailyProd << "), սահմանվեց լռելյայն՝ 10.0 կՎտ-ժ:\n";
```

```
        } else {
            dailyAverageProduction = dailyProd;
        }
    }

    void setLossPercentage(double loss) {
        if (loss < 0 || loss > 25) {
            lossPercentage = 0.0; // լռելյայն արժեք
            cout << "Զգուշացում: Կորուստների տոկոսը անվավեր է (" << loss << "), սահմանվեց լռելյայն՝ 0.0 (թույլատրված է 0-25%):\n";
        } else {
            lossPercentage = loss;
        }
    }

    // Getter-ներ
    string getStationName() const { return stationName; }
    int getWorkDays() const { return workDays; }
    double getDailyAverageProduction() const { return dailyAverageProduction; }
    double getLossPercentage() const { return lossPercentage; }

    // 3. Փոփոխող մեթոդ: Ավելացնել դրական թվով աշխատանքային օրեր
    void addWorkDays(int additionalDays) {
        if (additionalDays > 0) {
            workDays += additionalDays;
        } else {
            cout << "Սխալ: Ավելացվող օրերի քանակը պետք է լինի դրական թիվ:\n";
        }
```

```
    }

    // 4. Հաշվարկային մեթոդներ
    // Համախառն արտադրանք = օրեր * օրական արտադրանք
    double calculateGrossProduction() const {
        return workDays * dailyAverageProduction;
    }

    // Օգտակար արտադրանք = համախառն * (1 - տոկոս/100)
    double calculateUsefulProduction() const {
        double gross = calculateGrossProduction();
        return gross * (1.0 - lossPercentage / 100.0);
    }

    // 5. Ցուցադրման մեթոդ
    void display() const {
        cout << "----------------------------------------\n";
        cout << "Կայանի անունը: " << stationName << "\n";
        cout << "Օրերի քանակը: " << workDays << "\n";
        cout << "Օրական միջին արտադրանք: " << dailyAverageProduction << " կՎտ-ժ\n";
        cout << "Կորուստների տոկոս: " << lossPercentage << "%\n";
        cout << "Համախառն արտադրանք: " << calculateGrossProduction() << " կՎտ-ժ\n";
        cout << "Օգտակար արտադրանք: " << calculateUsefulProduction() << " կՎտ-ժ\n";
        cout << "----------------------------------------\n";
    }
};

int main() {
```

```
setlocale(LC_ALL, "Armenian");

// Ստեղծել 3 առանձին օբյեկտ՝ յուրաքանչյուրը տարբեր կոնստրուկտորով
cout << "=== 1. Առանձին օբյեկտների ստեղծում ===\n";

cout << "\n--- Օբյեկտ 1 (Լռելյայն կոնստրուկտոր) ---\n";
SolarStation station1;
station1.display();

cout << "\n--- Օբյեկտ 2 (3 պարամետրով կոնստրուկտոր) ---\n";
SolarStation station2("Արև-1", 15, 40.0);
station2.display();

cout << "\n--- Օբյեկտ 3 (Ստուգիչ օրինակ՝ 20 օր, 50 կՎտժ, 10% կորուստ) ---\n";
SolarStation station3("Արև-2", 20, 50.0, 10.0);
station3.display();

// Ստեղծել 5 օբյեկտից բաղկացած զանգված
const int SIZE = 5;
SolarStation stations[SIZE] = {
    SolarStation("Կայան Ա", 10, 30.0, 5.0),
    SolarStation("Կայան Բ", 20, 50.0, 10.0), // Ստուգիչ տվյալներ
    SolarStation("Կայան Գ", 25, 45.0, 15.0),
    SolarStation("Կայան Դ", 30, 60.0, 8.0),
    SolarStation("Կայան Ե", -5, -20.0, 35.0)  // Անվավեր տվյալների ստուգում
};

cout << "\n=== 2. 5 օբյեկտների զանգվածի ցուցադրում ===\n";
```

```
    for (int i = 0; i < SIZE; i++) {
        cout << "Զանգվածի ինդեքս " << i << ":\n";
        stations[i].display();
    }

    // Զանգվածի մեկ ընտրված օբյեկտի համար կանչել փոփոխող մեթոդը
    cout << "\n=== 3. Փոփոխության կիրառում զանգվածի 1-ին ինդեքսի (Կայան Բ) վրա ===\n";
    cout << "Մինչև ավելացումը (օրեր): " << stations[1].getWorkDays() << "\n";
    stations[1].addWorkDays(5); // Ավելացնում ենք 5 աշխատանքային օր
    cout << "Ավելացումից հետո:\n";
    stations[1].display();

    // Որոնում՝ ամենամեծ օգտակար արտադրանք ունեցող կայանը և ընդհանուր գումարը
    int maxIndex = 0;
    double maxUseful = stations[0].calculateUsefulProduction();
    double totalUsefulProduction = 0.0;

    for (int i = 0; i < SIZE; i++) {
        double currentUseful = stations[i].calculateUsefulProduction();
        totalUsefulProduction += currentUseful;

        if (currentUseful > maxUseful) {
            maxUseful = currentUseful;
            maxIndex = i;
        }
    }
```

```
    cout << "\n=== 4. Հաշվարկային արդյունքներ զանգվածի համար ===\n";
    cout << "Բոլոր կայանների օգտակար արտադրանքների գումարը: " << totalUsefulProduction << " կՎտ-ժ\n";
    cout << "\nԱմենամեծ օգտակար արտադրանքով կայանը (ինդեքս " << maxIndex << "):\n";
    stations[maxIndex].display();

    return 0;
}
```